

Retrieval-Augmented Generation (RAG)

Fundamentos, arquitectura, técnicas avanzadas y evaluación

Limitaciones de los LLMs

PROBLEMAS PRINCIPALES



Alucinaciones

Respuestas inventadas con total seguridad.



Información desactualizada

El conocimiento queda fijo en la fecha de entrenamiento.



Baja eficiencia

Reentrenar el modelo para parametrizar conocimiento nuevo es costoso.



Falta de profundidad

Dominios especializados quedan poco cubiertos.



Debilidad inferencial

Razonamiento limitado sobre hechos no vistos explícitamente.

REQUISITOS PRÁCTICOS PARA LA APLICACIÓN



Respuestas precisas

Específicas por dominio, no genéricas.



Actualización frecuente

Los datos cambian; el sistema debe reflejarlo.



Trazabilidad

Explicabilidad del contenido generado.



Costo controlable

En uso y despliegue.

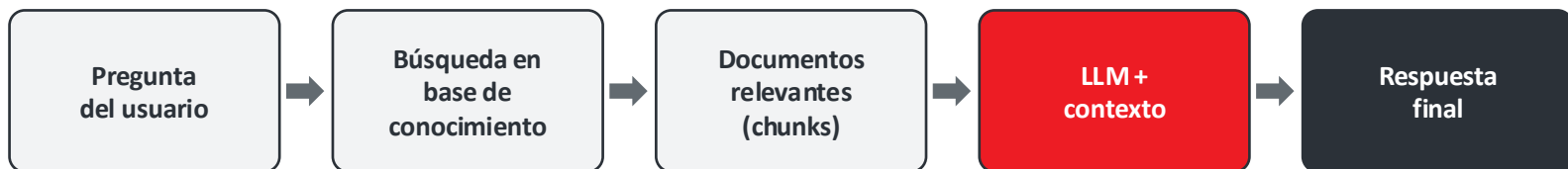


Protección de datos

Privacidad de la información consultada.

RAG: integrando recuperación y generación

Al responder preguntas o generar texto, el modelo recupera primero información relevante desde una gran colección de documentos. Luego, el LLM genera la respuesta en base a esa información — sin necesidad de reentrenar el modelo para cada tarea específica.



Especialmente útil en tareas intensivas en conocimiento — legal, médico, académico — donde la precisión y la actualidad de la información importan más que la fluidez por sí sola.

Componentes de RAG: Preprocesamiento

1

Construcción de la base de conocimiento

Transformar los datos en bruto en una knowledge base: embeddings vectoriales + metadatos para búsqueda eficiente.

2

Chunking (segmentación)

Dividir documentos largos en fragmentos pequeños y manejables — mejora la recuperación y evita problemas con textos extensos.

3

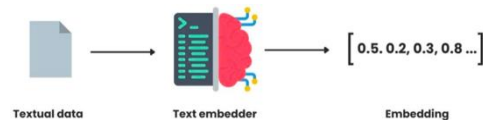
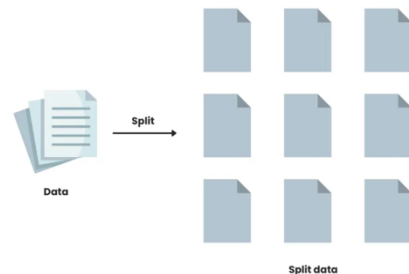
Generación de embeddings

Convertir cada fragmento en un vector numérico con un text embedder. La elección del modelo afecta directamente la calidad.

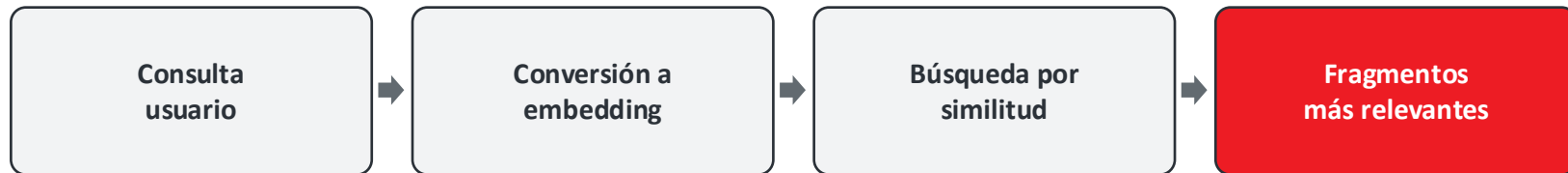
4

Almacenamiento en vector database

Guardar los embeddings en una base de datos vectorial que permite búsqueda rápida en espacios de alta dimensión.

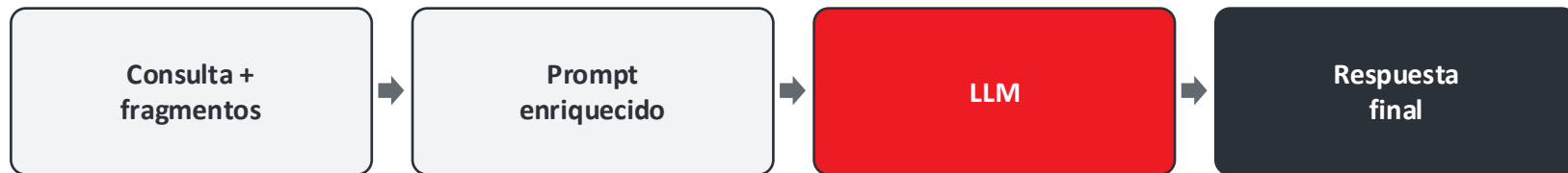


Componentes de RAG: Retrieval



- El sistema consulta la base de datos vectorial realizando una búsqueda por similitud entre la consulta del usuario y los embeddings almacenados.
- La consulta se convierte a un embedding usando el mismo text embedder (o uno entrenado en conjunto).
- Con un algoritmo de similitud (cosine similarity, dot product) se ordenan los fragmentos según su proximidad semántica.
- Se priorizan los fragmentos más relevantes para la intención del usuario.

Componentes de RAG: Generación

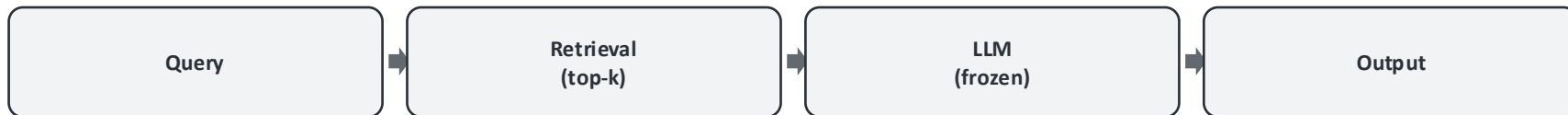


- Tras recuperar los fragmentos relevantes desde la base vectorial, el sistema construye un prompt enriquecido.
- El prompt combina la consulta del usuario y el contenido recuperado (chunks más relevantes).
- Este prompt guía al LLM en la generación de la respuesta final.
- Así, el modelo aprovecha conocimiento actualizado y específico, sin necesidad de reentrenarse.

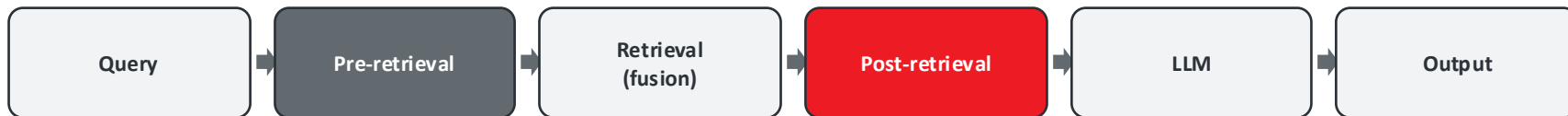
RAG Avanzado

Introduce mejoras antes y después de la recuperación: pre-retrieval optimiza la representación de documentos y del índice (embeddings más ricos, filtros); post-retrieval aplica re-ranking y selección inteligente de fragmentos.

NAIVE RAG (Tutorial 1)



ADVANCED RAG (Tutorial 2)



Mejoras en RAG



Pre-Retrieval Enhancements

- Granularidad de los datos: eliminar información irrelevante, asegurar consistencia y precisión factual
- Añadir contexto con anotaciones específicas de dominio
- Estructuras de índice optimizadas: ajustar tamaño de chunks, modificar rutas de indexación
- Incorporar información de estructura en grafo



Post-Retrieval Enhancements

- Reranking: reordenar pasajes según relevancia semántica
- Filtering: eliminar pasajes irrelevantes o redundantes (duplicados, análisis semántico)
- Summarization: resúmenes adaptativos para superar el límite de la ventana de contexto
- Expansion: enriquecer los pasajes recuperados con información relacionada

Optimización del Pipeline RAG



Hybrid Search

Combina búsqueda por palabras clave, búsqueda semántica y búsqueda vectorial.



Cognitive Backtracking

El sistema puede retroceder y reevaluar pasos previos de recuperación o generación, adaptándose a nueva información o feedback del usuario.



Estrategias diversas de consulta

Reformulación dinámica según retroalimentación; distintos tipos de consultas (hechos vs. opiniones); ajuste automático según el contexto de la conversación.

Desafíos del RAG Tradicional



Comprensión superficial

- Basado en medidas simples de similitud
- Puede fallar en captar matices y relaciones profundas



Redundancia y ruido

- Se entregan todos los chunks recuperados al LLM
- Información redundante o irrelevante → más alucinaciones



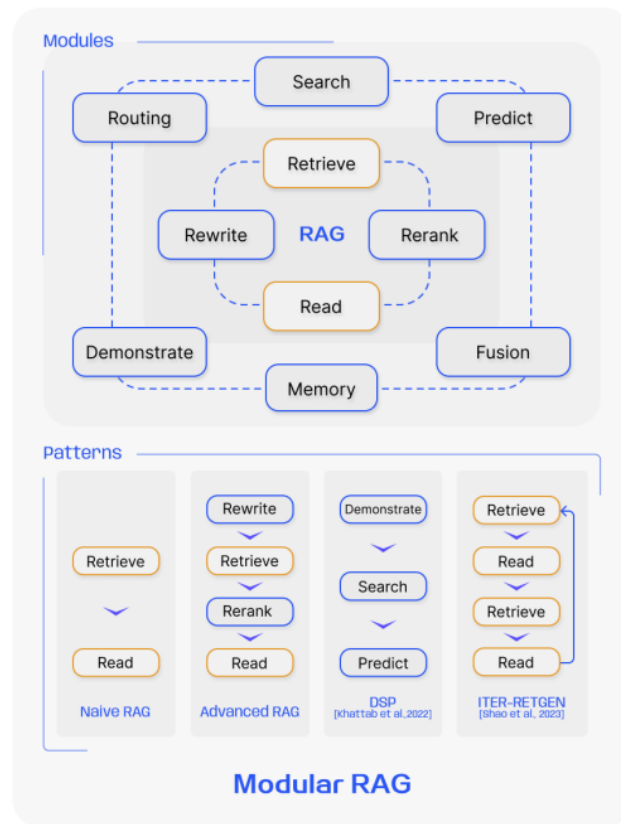
Proceso lineal rígido

- Flujo fijo "recuperar → generar"
- Dificultad con consultas complejas o multi-fuente

Aunque existen mejoras (re-ranking, query rewriting), la estructura lineal sigue siendo limitada — de ahí la necesidad de arquitecturas modulares.

Introducción a RAG Modular

- El esquema clásico "recuperar → generar" muestra limitaciones ante tareas complejas.
- La idea: descomponer la arquitectura en módulos independientes pero conectados.
- Cada módulo puede desarrollarse y optimizarse por separado, y reconfigurarse según la tarea.
- Metáfora LEGO: cada módulo es un bloque que se ensambla de distintas formas para construir un sistema adaptable y versátil.



Fuente: arxiv.org/pdf/2312.10997

Evaluación de RAG

R

Calidad de Recuperación

- ¿Se recuperan los documentos relevantes?
- Métricas: Recall@k, Precision@k, MRR

G

Calidad de Generación

- ¿La respuesta es correcta, coherente y útil?
- Métricas: Exact Match, F1, Accuracy, BLEU, ROUGE

faithfulness · answer relevancy

generation — qué tan fiel y relevante es la respuesta generada

context precision · context recall

retrieval — señal-a-ruido y cobertura del contexto recuperado

Cuándo usar Fine-tuning y cuándo RAG

F

Fine-Tuning (FT)

- Tareas estables en el tiempo, con dominio bien definido
- Adaptar estilo, tono o formato de salida
- Ej.: chatbots de servicio al cliente en un dominio específico
- Ventaja: respuestas consistentes y personalizadas
- Limitación: requiere reentrenar al actualizar la base de conocimiento

R

Retrieval-Augmented Generation (RAG)

- Tareas con datos dinámicos o en constante cambio
- Consultas abiertas: QA single/multi-hop, extracción, diálogo, code search
- Ventaja: acceso a información actualizada sin reentrenar; escalable a múltiples dominios
- Evaluación: calidad de recuperación y generación (EM, F1, Accuracy, BLEU, ROUGE)

Paradigmas avanzados de RAG

MÁS ALLÁ DEL RAG AVANZADO CLÁSICO

S

Self-RAG

- El propio modelo decide cuándo recuperar y qué recuperar (no siempre es necesario)
- Se autocritica con "reflection tokens": evalúa si el contexto recuperado es relevante y si la respuesta está soportada
- Útil cuando el costo de recuperar siempre es alto

✓

Corrective RAG (CRAG)

- Un evaluador liviano califica la calidad de lo recuperado (correcto / ambiguo / incorrecto)
- Si es insuficiente, corrige: refina la consulta, busca en la web, o descarta el contexto
- Reduce respuestas basadas en contexto irrelevante

A

Agentic RAG

- Un agente LLM planifica múltiples pasos de recuperación y usa herramientas (búsqueda, APIs, calculadora)
- Itera: recupera, evalúa, vuelve a recuperar si hace falta
- Generaliza la idea de "Cognitive Backtracking" vista antes

Frameworks de evaluación de RAG

HERRAMIENTAS PARA MEDIR, NO SOLO DEFINICIONES

Framework	Qué mide / cómo funciona	Notas
RAGAS	Faithfulness, answer relevancy, context precision/recall — usa un LLM auxiliar como juez	Open-source; se integra bien con LangChain/LlamaIndex
DeepEval	Las mismas métricas de RAGAS + G-Eval y métricas custom; test cases estilo pytest	El que usamos en el laboratorio del curso (Clase 3)
TruLens	"Feedback functions" configurables + tracing y dashboards de las llamadas al LLM	Fuerte en observabilidad / debugging de producción

En el Tutorial 3 (Evaluación de RAG) usamos DeepEval para comparar cuantitativamente el pipeline naive del Tutorial 1 contra el pipeline avanzado del Tutorial 2.

Buenas prácticas: chunking y embeddings

GUÍA PRÁCTICA PARA EL PRE-RETRIEVAL

Heurísticas de chunking

- 256-512 tokens: FAQs, tickets, mensajes cortos
- 512-1024 tokens: artículos, documentación técnica
- Overlap típico: 10-20% del chunk_size
- Splitting semántico (por encabezados/párrafos) > corte fijo por caracteres cuando el documento tiene estructura
- Verifica el largo en tokens del modelo de embeddings, no solo en caracteres (Clase 2)

Elegir un modelo de embeddings

Modelo	Tipo	Uso
OpenAI text-embedding-3	API	Rápido de integrar, buen default
Cohere embed-v3	API	Fuerte en multilingüe
BGE / E5 (HF)	Open-source	Gratis, corre en CPU (Clase 1)

RESUMEN

Lo que vimos en RAG Simple y Avanzado

1

Pipeline base

Chunking → embeddings → vector store → retrieval → generación

2

Pre-retrieval

Chunk size, granularidad, estructura del índice

3

Post-retrieval

Fusion retrieval (BM25+vectorial), reranking, filtrado

4

RAG modular

Módulos reconfigurables en vez de un pipeline rígido

5

Evaluación

Retrieval (Recall@k, MRR) vs generación (faithfulness, relevancy)

6

RAG vs Fine-tuning

Datos dinámicos → RAG; estilo/dominio fijo → fine-tuning

→ Próxima clase: cuando las preguntas requieren conectar entidades entre sí, RAG vectorial no basta — entra GraphRAG.